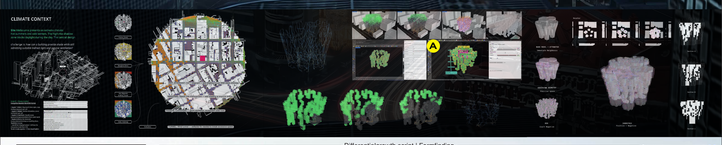
"A building is not shaped by its walls — it is shaped by where light breaks through them." "What if the crack in the building is not a failure — it is the architecture?

ARCHITECT IN THE CRACK

This project investigates how a research building for machine learning can simultaneously provide a comfortable learning environment and an isolated refuge from Melbourne's extreme urban climate — through architecture grown by multi-agents across a pipeline of five computational scripts, not designed conventionally.

- Script stages
- Room locations



THE CRACK

The climate field is applied to the base massing to generate attractor points, which inform voxel distribution in the next stage. All voxels are populated with attractor data drawn from the urban context.

The Space Colonization Algorithm (SCA) was then applied to carve a rift through the mass, mimicking canyon and cliff formations found in nature, channelling indirect light and wind deep into the building. This is the crack — the central architectural move.

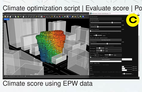
Script - Colonization_Algorithm

Description & How It Works: Models how roots or veins branch—agents "colonize" free space by spawning branches toward nearest unvisited space nodes. Outputs branching tree curves (not volumes).

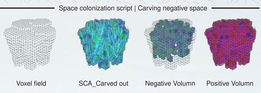
Differentialgrowth script | Formfinding

Climate optimization script | Evaluate score | Points cloud attractor

Space colonization script | Carving negative space



Climate score using EPW data



Final form | Voxel field | SCA_Carved out | Negative Volumn | Positive Volumn

Discrete Element Study

The discrete element investigation focuses on timber and metal, studying three joinery conditions: interlocking, tongue and groove, and metal plate and spike connectors. Parts were manually assembled into chunks to understand how components fit together under different tectonic conditions — floor, wall, column, and facade. This manual process established the assembly logic before scripting. Hierarchy: Parts → Chunks → Architecture. Three output modes: Manual assembly / Chunks placed on field / Procedurally generated on the field.

Timber tectonic | Groove and tongue

Timber tectonic | Interlocking

Chunks iterations (outputs)

Metal plate and spike connector

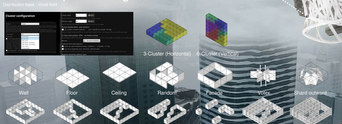
Chunks iterations (outputs)

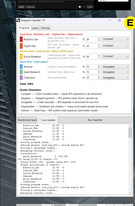
DISTRIBUTION DISCRET ELEMENT

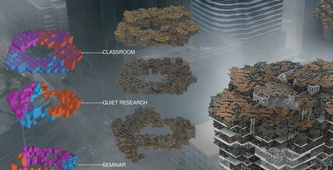
Distribution / Aggregation

The distribution agent reads the voxel field as its base and populates discrete element chunks across it, responding to attractor values and programmatic zones. Through the combination of Voronoi clustering and K-means, alongside multi-agent room logic, the script drives variation in cluster type, room resolution, and tectonic placement across the building.

Distribution | Tectonic logics







PROGRAM CLASIFIER SCRIPT

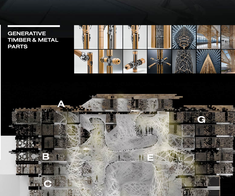
How It Works: Tags voxel clusters into one of seven program types (Robotics Lab, Digital Fab, Makerspace, Seminar, Social Space, Quiet Research, Learning Refuge) based on cluster size/shape and multi agent Voronoi + K-means clustering.

Agents: Voxel clusters (connected components)

Behaviour: Evaluate clustering rules → assign program label

Together with the Program Classifier (E) and Distribution Discrete Elements (D) scripts, each function was allocated different logics — including assignment, tectonic placement and orientation, and Voronoi cluster logic. We can then configure a variety of architectural outcomes to achieve the desired effect. For instance: wall thickness responding to the climate attractor, variable wall resolution (Low-High), volume height, circulation separation, or open floor plan.

GENERATIVE TIMBER & METAL PARTS

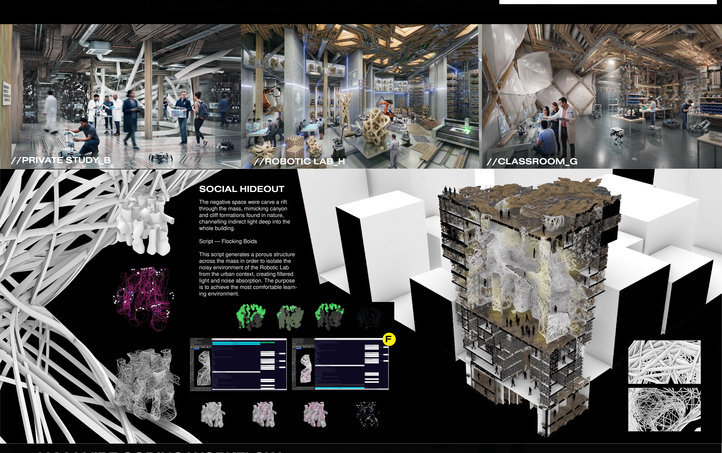


SOCIAL HIDEOUT

The negative space were carved out through the mass, mimicking canyon and cliff formations found in nature, channelling indirect light deep into the whole building.

Script — Flocking Boids

This script generates a porous structure across the mass in order to isolate the noisy environment of the Robotic Lab from the urban context, creating filtered light and noise absorption. The purpose is to achieve the most comfortable learning environment.



LLM | VIBE CODING WORKFLOW

Multi-agents | MCP | Markdown | Feedback loop | Rhino Python

Text prompts, image references, and MCP tools feed into an LLM agent, generating successive script iterations. Failed outputs return as screenshot feedback, closing the loop.

// ROOFTOP_A

Recreation of Green space